HCCAI: A Hardware-Rooted, Evidence-Gated Control Plane for Verifiable AI-Agent Execution

Formal Safety Invariants, Threshold Governance, and Production Validation of the v6.9 Artifact

AI-Based Cryptography Research Team
J1729Labs

October 2026

Abstract

Autonomous AI agents increasingly execute actions that cross trust boundaries: they access protected data, consume budgets, invoke external tools, and may trigger irreversible state transitions. Application-layer authorization alone cannot guarantee that execution integrity survives process crashes, signer compromise, replay, operator changes, and deployment-site uncertainty. This paper presents HCCAI, a production-oriented control plane that binds AI-agent execution to a single conjunctive admission predicate spanning cryptographic authority, policy validity, hardware-rooted attestation, threshold governance, append-only evidence, and deterministic replicated state.

We formalize eight safety invariants — execution admission, replay-safe nonce consumption, attestation binding, quorum-certificate intersection, cross-epoch transition safety, threshold nonce safety, commit/apply separation, and evidence-gated promotion — and give proofs under explicit cryptographic and storage assumptions, stated once as a shared assumption set rather than repeated per theorem. We distinguish implemented guarantees from the stronger consensus or hardware claims the system intentionally does not make.

Validation is artifact-backed: the v6.9 package passes the full Go functional and race-detector suites, static analysis, production builds, C/OpenSSL signer compilation, 100 repeated execution-receipt concurrency runs, and 20 race-enabled repetitions. The package contains 79 executable binaries and a signed, fail-closed site-execution path. The build host lacks a physical TPM resource manager, IMA runtime logs, and an HSM provider, so hardware-in-the-loop promotion is correctly blocked (exit code 2) — a negative result we treat as evidence, not a gap in the evidence.

Keywords: AI agent security, hardware-rooted attestation, TPM 2.0, IMA, threshold signatures, FROST, Byzantine fault tolerance, deterministic state machines, fail-closed deployment.

1 Introduction

AI-agent systems pose a security problem that ordinary request/response applications do not: an agent maintains long-lived context, repeatedly invokes tools, spends budget, delegates authority, and produces a sequence of externally visible effects. Authorization checked only at session start — or evidence that can be rewritten after the fact — collapses under replay, stale authority, partial compromise, or crash recovery. HCCAI's objective is therefore not to decide whether one call is allowed, but to make the entire execution *transition* verifiable, end to end.

The design is organized around a single idea: the admission predicate must be conjunctive and fail-closed at every layer it touches, from a signed authority check up through hardware

attestation, threshold-issued authority, and deterministic replicated state. A second, equally load-bearing idea is that claims must stay bounded by what was actually executed. A build host with no TPM must not be reported as having passed a physical TPM acceptance test; a majority-commit primitive must not be mislabeled as full asynchronous consensus. The production artifact is built around explicit claim boundaries and fail-closed promotion gates so that "not tested" can never silently read as "passed."

This paper contributes: (1) a unified architecture spanning authority, proof, policy, hardware attestation, threshold issuance, evidence transparency, controller replication, and deterministic execution; (2) eight formalized safety invariants with proofs under a shared, explicitly stated assumption set; (3) a description of the production path from local multi-process safety through hardware-rooted, distributed operation to customer-site rollout; (4) artifact-backed validation results from the v6.9 package; and (5) a precise account of residual limitations, most notably the build host's unavailable physical HIL and the gap between implemented consensus-grade primitives and full asynchronous consensus.

2 System Model

Execution as a state transition. An agent execution request is a tuple $R = (a, x, \pi, p, n, b, c, \tau)$, where a is authority, x the requested operation, π a proof object, p the current policy state, n a freshness nonce, b budget state, c a capability, and τ contextual trust evidence such as hardware attestation. The core enforcement invariant generalizes the familiar policy-gateway check into a conjunction over independent validity predicates:

$$\text{Execute}(R) = A \wedge P \wedge L \wedge F \wedge B \wedge C \wedge T \tag{1}$$

where A, P, L, F, B, C, T denote authority, proof, policy, freshness, budget, capability, and trust-evidence validity respectively. In later layers T itself decomposes into software provenance, TPM evidence, threshold-authority admission, and deployment evidence — the conjunction nests, but it never relaxes: every added layer is another term ANDed into the predicate, never a term that can substitute for one already there.

Definition 2.1 (Fail-closed execution)**.** *An execution path is* fail-closed *if any missing, malformed, stale, unverifiable, or policy-inconsistent mandatory input maps to* $\text{Execute}(R) = 0$*, and no combination of present inputs can convert that* 0 *into acceptance by omission of the missing one.*

This definition is what makes §4.8's promotion gate and §5's physical-HIL boundary the same mechanism applied at different layers, rather than two unrelated design choices.

Adversary and trust assumptions. The adversary may replay network requests, race concurrent processes, mutate local files without signing keys, present stale certificates or inconsistent controller state, crash processes mid-write, compromise a minority of quorum participants, and attempt to reuse old epoch keys or deployment evidence. The system does *not* assume protection against full threshold compromise, breakage of the underlying signature or hash primitives, or malicious hardware below the configured root of trust.

All proofs in §4 share one assumption set, stated once here rather than re-derived per theorem:

> Σ **(shared assumptions).** (Σ1) SHA-256 is collision-resistant where used for hashing or domain separation. (Σ2) Ed25519, ECDSA, and FROST signatures are unforgeable under their stated key-generation assumptions. (Σ3) TPM measurements are correct whenever physical TPM evidence is actually present and verified. (Σ4)

POSIX locking and `fsync` on the supported local filesystems behave per their documented semantics. ($\Sigma 5$) Each subsystem's quorum threshold is honestly parameterized (e.g., $n \geq 3f + 1$ for the Byzantine committee).

Every theorem below is implicitly "under Σ"; where a proof needs a subset of Σ we name it for precision, not because the others don't hold.

3 Architecture

The production architecture (Figure 1) routes every agent/tool request through a single gateway that composes six planes: an **authority and capability plane** that authenticates requests against domain-separated messages and enforces exact-request idempotency and one-shot capability consumption, independently re-checked at the protected resource so a compromised gateway alone cannot bypass authorization; a **proof-provider boundary** that invokes external proof systems (including zk-FIDNA) through a hardened process boundary — absolute paths, permission checks, bounded output, explicit timeouts, request-digest binding — so the trust model never collapses into an unconstrained shell call; a **hardware-rooted attestation plane** (§5) that verifies TPM 2.0 quotes, AK identity, Secure Boot state, and IMA measurements, and can cryptographically bind a successful attestation into issued authority; a **threshold and Byzantine-hardened authority plane** (§6) built on FROST threshold signing with mutual-TLS transport, durable nonce burning, and epoch-certified membership change; an **evidence and replication plane** (§6) of HA controllers maintaining a hash-linked WAL, signed majority commit certificates, and a deterministic state machine with a hard separation between *commit* and *apply*; and a **deployment and site-acceptance plane** (§7) that requires independently signed evidence for HIL, disaster recovery, canary rollout, and rollback rehearsal before a site can be promoted.

The throughline across all six planes is that each one independently fails closed under Definition 2.1, and each feeds a term into the nested conjunction of §2 rather than acting as an alternate path to admission.

4 Formal Safety Invariants

This section states and proves the eight predicates that, together, constitute HCCAI's safety argument. They are presented in the order an execution actually passes through them — admission, then attestation, then quorum formation, then epoch change, then nonce handling, then state application, then promotion — so that the composition in §4.8 reads as the product of everything proved before it.

4.1 Replay-safe durable admission

The first thing any execution must clear is that it has not already happened. Let N_a denote the set of nonces already committed for authority a.

Theorem 4.1 (Single-consumption freshness)**.** *If nonce insertion and budget reservation are performed within one serialized durable transaction, and a request is admitted only when $n \notin N_a$, then no two successful executions under the same (a, n) can both consume budget.*

Proof. Suppose two distinct commits E_1, E_2 both use (a, n). Serialization ($\Sigma 4$) totally orders them; say $E_1 \prec E_2$. After E_1 commits, $n \in N_a$ by construction. The admission precondition for E_2 then evaluates to false, contradicting that E_2 commits. Exact-request idempotency may legitimately return E_1's already-committed result on a retry, but this returns a cached outcome — it does not open a second budget transition. ∎

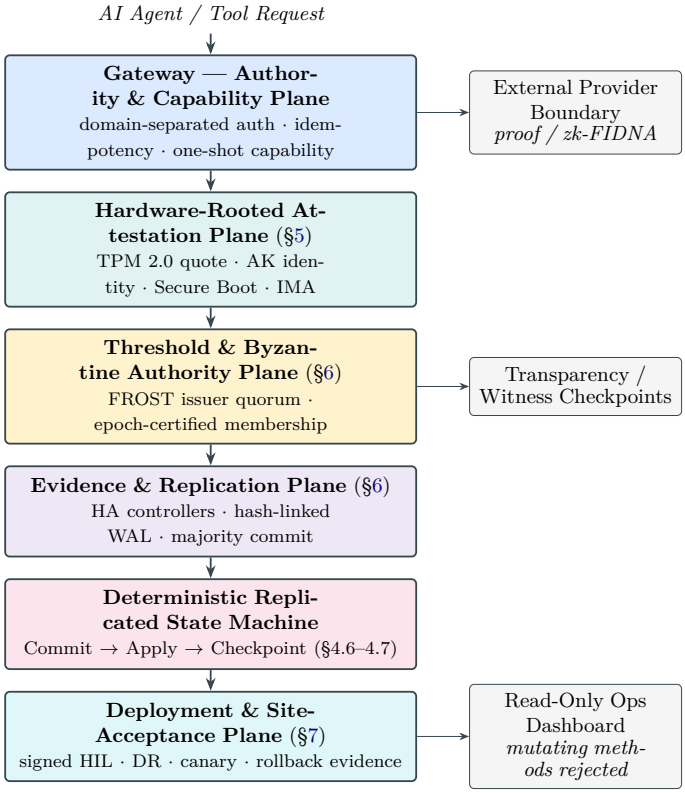


Figure 1: Production architecture of the HCCAI v6.9 artifact. Each plane independently fails closed (Definition 2.1) and feeds one more conjunct into Execute(R) — none substitutes for another. Hardware evidence is a mandatory external input to hardware-gated promotion; it is never synthesized by the build environment itself.

This is the base case the rest of §4 builds on: every later invariant assumes that a single admitted request corresponds to a single state change, which is exactly what Theorem 4.1 guarantees.

4.2 Attestation binding

Once a request is known to be fresh, its binding to *this* execution environment must be non-substitutable. Define the attestation admission digest $D_A = H(\text{domain}, D_E, D_R, D_C, D_W, D_{NV})$ over evidence D_E, attestation receipt D_R, checkpoint state D_C, witness-quorum evidence D_W, and anti-rollback anchor D_{NV}.

Proposition 4.2 (Non-substitutability of bound attestation)**.** *If an authority signature covers D_A, replacing any one bound component while preserving that signature requires either a hash collision (violating $\Sigma 1$) or a signature forgery (violating $\Sigma 2$).*

Proof. Changing any component of the tuple changes D_A except on a collision ($\Sigma 1$). Verification checks the signature against the originally signed digest, so a changed digest requires a new valid signature over it ($\Sigma 2$). ■

The practical reading: an attacker who can swap, say, the IMA measurement log after the fact without touching the signature gets caught by this proposition, not by a separate integrity check bolted on afterward — the binding *is* the integrity check.

4.3 Quorum-certificate intersection

Threshold authority depends on any two valid certificates sharing at least one honest signer. For a Byzantine committee of size $n \geq 3f + 1$, certificates require $2f + 1$ validator signatures.

Lemma 4.3 (Byzantine quorum intersection)**.** *Any two sets $Q_1, Q_2 \subseteq V$ with $|Q_i| \geq 2f + 1$ intersect in at least $f + 1$ validators.*

Proof. $|Q_1 \cap Q_2| \geq |Q_1| + |Q_2| - |V| \geq (2f + 1) + (2f + 1) - (3f + 1) = f + 1.$ ■

Corollary 4.4. *Since at most f validators are Byzantine, any two valid $2f + 1$ certificates share at least one honest validator — the fact that underwrites finality certificates, timeout/view certificates, and the epoch-transition safety proved next.*

4.4 Cross-epoch transition safety

Validator-set changes are where naive threshold systems leak safety, because a stale quorum signing an old epoch is indistinguishable from a legitimate one unless the transition itself is certified. A transition from epoch e to $e + 1$ requires joint old/new certification, $\text{JointCert} = Q_{\text{old}}^{2f_o+1} \wedge Q_{\text{new}}^{2f_n+1}$, together with durable monotonicity $e_{\text{new}} = e_{\text{old}}+1$ and $\text{Prev}_{\text{new}} = \text{Digest}(M_{\text{old}})$.

Theorem 4.5 (Serialized epoch rollback resistance)**.** *Under unforgeable joint certificates ($\Sigma 2$) and monotonic durable epoch storage ($\Sigma 4$), an accepted transition cannot be replaced by a lower or parallel epoch without either violating the local monotonicity check or forging a certificate or chain digest.*

Proof. A lower epoch directly violates the strict-increment condition and is rejected. A distinct manifest proposed at the same next epoch necessarily has either a different manifest digest or a different predecessor digest from the accepted one; the durable store rejects same-or-lower epochs and any broken predecessor chain. Bypassing both checks requires forging authenticated transition material ($\Sigma 2$) or writing outside the assumed durable-storage model ($\Sigma 4$) — both excluded by assumption. ■

4.5 Threshold nonce safety

Within a single epoch, the FROST signing path burns its round-2 nonce durably before releasing a signature share.

Proposition 4.6 (Crash-safe nonce non-reuse)**.** *If every nonce is durably marked burned before its round-2 share is released, a process crash between burn and share-delivery can degrade liveness but cannot cause compliant software to safely reissue that nonce.*

Proof. On restart, nonce allocation consults the durable vault; a burned record is categorically ineligible for reallocation. The crash may discard the in-flight share and force a fresh signing session, but reuse of the burned nonce is excluded by construction of the allocator, independent of what the crashed process was doing when it died. ■

4.6 Commit/apply separation

Durable logging and state-machine application are deliberately decoupled so that a crash between the two never exposes a logically inconsistent read. Let c be the durable commit index and a the applied index.

Definition 4.7 (Readable state invariant)**.** *A local materialized state is readable only when $0 \le a \le c$; a read barrier certified at index i is satisfied only when $a \ge i \wedge c \ge i$.*

Theorem 4.8 (No uncommitted materialization)**.** *If the state machine applies records strictly in increasing index order and only for $j \le c$, no readable materialized state ever includes a command with index greater than the durable commit index.*

Proof. By induction on apply steps. Base case: initially $a \le c$. Inductive step: each apply chooses $j = a + 1$ and checks $j \le c$ before applying; after application $a' = j \le c$, preserving the invariant. A failed application is quarantined rather than advancing a, so the invariant is never violated even on partial failure. ■

4.7 Deterministic state hashing

Replica agreement requires that independently computed checkpoints be byte-identical regardless of implementation-level map iteration order. Over a materialized map of pairs (k_i, v_i), the checkpoint hash is $D_S = H(\text{domain} \,\|\, \langle k_1, v_1\rangle \,\|\, \cdots \,\|\, \langle k_m, v_m\rangle)$ with keys sorted lexicographically and values length-delimited.

Theorem 4.9 (Map-order independence)**.** *Two replicas with identical logical key-value maps compute identical D_S regardless of language-level map iteration order.*

Proof. Both replicas sort the same finite key set under the same total order before hashing, and length-delimited encoding is deterministic, so the byte sequence fed to H is identical across replicas by construction — map iteration order never enters the computation. ■

4.8 Evidence-gated production promotion

The invariants above compose into the single gate that actually controls whether a customer site goes live. Let $G = \{g_P, g_H, g_D, g_C, g_R\}$ be the mandatory site gates for preflight, hardware HIL, disaster recovery, canary, and rollback rehearsal, and define $\text{Promote} = \bigwedge_{g \in G} \text{ValidSignedEvidence}(g)$.

Theorem 4.10 (No promotion by omission)**.** *If the site runner initializes every gate to* false *and transitions to* PROMOTED/ACCEPTED *only when all $g \in G$ verify, then the absence of any mandatory evidence cannot produce promotion.*

Proof. The promotion predicate is a pure conjunction. For any missing gate g, ValidSignedEvidence(g) = 0 by initialization, so the conjunction evaluates to 0 regardless of the other gates. Durable phase-transition checks additionally prohibit any direct jump that skips a gate's initialization. ■

Theorem 4.8 is Definition 2.1 instantiated at the deployment layer: it is the formal reason §5's missing-TPM build host is *correctly* blocked rather than a bug to work around.

5 Hardware-Rooted Attestation

TPM identity and measured boot. The attestation plane verifies TPM 2.0 quote semantics, AK public-key/name relationships (and AK X.509 relationships where configured), EK certificate identity, and TPM credential-activation workflows, with a possession challenge that prevents a remote party from satisfying an AK identity check merely by copying public material. IMA validation covers binary and ASCII measurement-log handling, per-bank PCR replay, key/signature semantics, and cross-source consistency; Secure Boot variables (`PK`, `KEK`, `db`, `dbx`) are parsed and bound to policy, so software state becomes part of authority admission rather than a side-channel monitoring signal. Rollback anchors — TPM clock/reset/restart data and signed NV lifecycle anchors — detect stale lifecycle state, and external witness checkpoints add an independent record of accepted state.

The physical-HIL claim boundary. The v6.9 build host reports `tpm_resource_manager: false, ima_binary_log: false, ima_ascii_log: false`, exiting with code 2. This is not a failed software build; it is the fail-closed environmental result that Theorem 4.8 predicts. The physical-HIL claim stays unasserted until the identical acceptance runner executes on an approved machine carrying a TPM resource manager, IMA runtime logs, and a configured HSM provider — the negative result is itself part of the security evidence, not a gap in it.

6 Threshold Governance and Evidence Replication

Threshold signing and ceremony hardening. The threshold subsystem runs a FROST-style P-256/SHA-256 signing core over mutually authenticated TLS, with proactive refresh, compromise-recovery primitives, dealer complaint handling, and signed ceremony transcripts whose sender sequence/digest chains make equivocation externally demonstrable — Merkle transcript inclusion proofs bind the signed broadcast object to both its claimed digest and leaf hash. The reference scalar backend uses `math/big` and is **not** claimed constant-time; side-channel-sensitive deployments require a reviewed constant-time or HSM-backed provider. Finality uses a fixed validator epoch with $n \geq 3f + 1$ and $2f + 1$ certificates (§4.3–4.4), deterministic leader selection, and a durable finality guard that rejects a second finalized value for the same slot; membership changes are serialized by epoch with independent old/new quorums and generation-monotonic key rotation.

Evidence controllers and replicated state. Field results are signed by agent keys and bound to tenant, site, job, release, target, digest, and size; the central evidence store accepts identical replays idempotently but rejects divergent content at the same immutable identity. Controller membership is epoch-certified, remote jobs use leases with monotonic fencing tokens, and a stale controller that regains network access cannot reuse an old fencing token once a field agent has accepted a higher one. The controller WAL forms a predecessor-hash chain with signed replica acknowledgements over epoch, index, term, and record digest; a majority certificate advances the durable commit index (§4.6), torn final appends are recoverable, but corruption of a complete intermediate record is treated as failure rather than silently truncated

away. Snapshot compaction binds last-included index/term plus head and state digests so post-compaction WAL entries continue from a single authenticated history, and an artifact-layer $k{+}1$ XOR parity mode tolerates single-shard loss — explicitly not Reed-Solomon or arbitrary multi-shard recovery.

7 Production Deployment and Operations

Read-only operations surface. The commercial dashboard exposes engine version, commit/applied indices, state hash, quarantine count, attestation and threshold-authority posture, GA readiness, deployment phase, and rollout decision, but rejects every mutating HTTP method — security-state changes remain CLI/governance operations, which bounds the blast radius of a compromised browser session to read access only.

Preflight, canary, and the site runner. Production preflight checks Linux host assumptions, expected binaries and permissions, physical TPM, IMA runtime files, and HSM module configuration; the acceptance report binds evidence paths, SHA-256 digests, and signed results. Rollout plans are signed and specify release digest, targets, canary count, and health policy, with canary promotion gated on healthy percentage, error rate, latency, and customer acceptance, and rollback rehearsal itself a signed gate rather than an optional step. The one-command site runner composes plan verification, preflight, and HIL/DR/canary/rollback evidence verification into the durable site-state transition of Theorem 4.8; a development host lacking required hardware correctly remains in PREFLIGHT/BLOCKED.

8 Experimental Validation

Method. Validation spans six classes: full functional regression (`go test ./...`), full data-race regression (`go test ./... -race`), static analysis (`go vet ./...`), build validation across all Go binaries plus the C/OpenSSL signer, adversarial/negative testing (stale quorums, mutated evidence, insufficient signatures, rollback attempts, double-finality attempts, stale fencing tokens, unsupported mutations), and site acceptance (physical-hardware probing and fail-closed promotion). The final v6.9 hardening pass additionally repeated the execution-receipt concurrency path 100 times after adding shared read locks around verification, plus 20 further race-detector repetitions.

Results. Table 1 reports release-level results for the final package.

Earlier checkpoints form a consistent progression rather than isolated snapshots: v6.0 (GA baseline) rejected protocol downgrade, signed-baseline mutation, provenance/artifact mutation, upgrade rollback, and DR epoch rollback/fork across 8/8 deterministic fault-campaign cases; v6.5 (controller durability) verified majority commit, rejected minority commit, recovered a torn final WAL append, rejected snapshot-anchor mutation, passed old/new joint-majority membership, recovered a single XOR-protected shard, and rejected multi-shard loss and double-commit failover; v6.6 (commercial final) passed 10 race-enabled repetitions each of the replicated-state-machine suite and the dashboard suite, with the dashboard rejecting a mutating POST with HTTP 405. The build-host hardware probe produced `phase=PREFLIGHT, status=BLOCKED, preflight_ready=false` with TPM, IMA ASCII, IMA binary, and HSM-module checks all failing — precisely the promotion behavior Theorem 4.8 requires when physical trust prerequisites are absent.

Validation item	Result
Signed site execution plan	PASS
Signed HIL/DR/CANARY/ROLLBACK evidence binding	PASS
Artifact SHA-256 revalidation	PASS
Durable site state chain	PASS
Fail-closed production preflight	PASS
Local no-hardware site validation	BLOCKED as expected (exit 2)
Execution-receipt concurrency, 100 repetitions	PASS
Execution-receipt package, race detector, 20 repetitions	PASS
Full `go test ./... -count=1`	PASS
Full `go test ./... -race -count=1`	PASS
`go vet ./...`	PASS
Full production build	PASS
C/OpenSSL signer (`-O2 -Wall -Wextra -Werror -lcrypto`)	PASS
Executable binaries in final build	79
Physical TPM/IMA/HSM acceptance on build host	NOT AVAILABLE

Table 1: v6.9 release validation summary.

9 Security Analysis and Comparison

Replay, signer compromise, and split-brain. Nonce uniqueness, durable idempotency, and resource-side capability verification jointly prevent a retry storm from becoming repeated budget consumption (Theorem 4.1), though the argument depends on transactional serialization of the selected backend — multi-host operation needs the distributed/database path, not shared uncoordinated filesystems. Minority signer compromise is addressed by threshold quorums and key-identity pinning (§4.3); complete threshold compromise remains explicitly outside the threat model, and proactive refresh reduces but does not eliminate the exposure window of already-compromised shares. Majority commit certificates, quorum-observed fork repair, fencing tokens, and site-failover checks reduce split-brain risk, but the controller is described as implementing consensus-*grade* durability primitives, not a proof-complete Raft, Paxos, or asynchronous BFT protocol.

Evidence mutation and dashboard risk. Signed evidence binds target/release identity and artifact digests; immutable stores reject divergent replacement at the same identity, and hash-linked logs make removal or reordering detectable from a trusted checkpoint. The operations dashboard is deliberately not an administrative surface — it exposes only reads — so browser or session compromise cannot reach security-sensitive transitions, which stay behind signed CLI/governance workflows.

Comparison with conventional designs. Table 2 is architectural, not a benchmark claim: it is meant to show which controls HCCAI combines that are conventionally deployed separately.

10 Claim Boundaries, Reproducibility, and Conclusion

What is not claimed. Physical-HIL success is not claimed on this build host, which lacks a physical TPM resource manager, IMA runtime logs, and a configured HSM provider. The reference P-256 scalar implementation is not claimed constant-time. The controller plane implements signed majority commit, WAL durability, snapshot compaction, joint-majority membership transition, leases/fencing, and quorum repair — it is not presented as a complete formal implementation of Raft, Paxos, or fully asynchronous BFT consensus. The $k{+}1$ XOR erasure scheme tolerates exactly one missing shard and is not equivalent to Reed-Solomon multi-erasure

Property	API Auth	Service Mesh	Attestation-only	HCCAI
Per-action policy binding	partial	partial	no	yes
Replay/budget durable semantics	app-specific	no	no	yes
Hardware state in authority issuance	no	no	yes/partial	yes
Threshold issuance	uncommon	no	no	yes
Execution receipt chain	uncommon	telemetry	attestation only	yes
Cross-epoch signer guard	no	no	no	yes
Commit/apply separation	app-specific	no	no	yes
Signed site promotion gates	no	no	no	yes
Fail-closed absent HIL	app-specific	no	verifier-specific	yes

Table 2: Architectural comparison with conventional designs.

coding. Cryptographic soundness of any deployment-specific external proof system remains a property of that provider, not of HCCAI's hardened invocation boundary around it. Organizational key custody, HSM administration, certificate issuance, and physical provisioning remain external operational controls outside the software artifact.

Reproducibility. The v6.9 artifact ships source, build-produced command binaries, per-version validation records, SHA-256 manifests, deployment templates, and production acceptance scripts. The core reproduction path is

```
go test ./... -count=1 && go test ./... -race -count=1 && go vet ./...
&& make build && make signer
```

after which site acceptance (`hccai-prod-site-run <signed-plan-and-evidence>`) remains hardware-dependent and must stay blocked unless preflight and all signed HIL/DR/canary/rollback evidence verify.

Conclusion. HCCAI's central engineering lesson is that verifiable AI execution is a property of a *chain* of invariants, not of any single cryptographic primitive: a TPM quote is insufficient if stale authority can be replayed, a threshold signature is insufficient if ceremony nonces can be reused after a crash, a durable log is insufficient if committed entries are applied nondeterministically, and a passing test suite is insufficient if the deployment process can relabel missing hardware as accepted. HCCAI addresses this by treating authority, hardware evidence, durability, and deployment status as one connected security object under the single fail-closed definition given in §2. The system's refusal to assert physical-HIL success on a host without TPM/IMA/HSM support is accordingly not a missing metric but confirmation that the promotion boundary behaves exactly as specified — a commercially complete software engine with hardware promotion correctly blocked, rather than a fabricated physical-GA claim.

A Acceptance Predicate Summary

$$\begin{aligned}\text{AcceptSite} = \text{PlanSigValid} \wedge \text{PreflightReady} \wedge \\ \text{HILEvidenceValid} \wedge \text{DREvidenceValid} \wedge \\ \text{CanaryEvidenceValid} \wedge \text{RollbackEvidenceValid} \wedge \\ \text{ArtifactDigestsMatch} \wedge \text{StateTransitionValid}\end{aligned}$$

On the v6.9 build host, PreflightReady = 0 because physical TPM/IMA/HSM prerequisites are absent, so AcceptSite = 0 independent of all later evidence — the conjunctive structure of §2 and §4.8 applied literally.

B Artifact Validation Record

Checkpoint	Recorded result
v6.0 GA baseline	Focused race ×10 PASS; downgrade/mutation/rollback attacks rejected; signed release/provenance chain verified
v6.4 HA controller	Focused race ×10 PASS; membership quorum, catch-up, fork repair, immutable replication, audit chain validated
v6.5 durability	Majority commit PASS; minority reject; WAL recovery PASS; snapshot compaction PASS; joint-majority membership PASS; single-shard recovery PASS; double-commit failover reject
v6.6 commercial final	RSM race ×10 PASS; dashboard race ×10 PASS; full regression/race/vet/build PASS; 69 executables; dashboard POST rejected
v6.7 deployment	Deploy/accept race ×10 PASS; 72 executables; preflight BLOCKED on missing TPM/IMA/HSM
v6.8 rollout	Rollout tests and race ×10 PASS; full regression/race/vet/build PASS; 75 executables; hardware preflight BLOCKED
v6.9 final site runner	Signed plan/evidence PASS; artifact revalidation PASS; durable site-state PASS; receipt concurrency ×100 PASS; receipt race ×20 PASS; full regression/race/vet/build PASS; 79 executables; local site BLOCKED as expected

Table 3: Artifact validation record across checkpoints.

References

[1] Trusted Computing Group, *TPM 2.0 Library Specification*.

[2] Linux kernel documentation, *Integrity Measurement Architecture (IMA)*.

[3] C. Komlo and I. Goldberg, "FROST: Flexible Round-Optimized Schnorr Threshold Signatures," *Selected Areas in Cryptography*, 2020.

[4] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm (Raft)," *USENIX ATC*, 2014.

[5] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," *OSDI*, 1999.

[6] NIST, *SP 800-193: Platform Firmware Resiliency Guidelines*, 2018.

[7] D. J. Bernstein et al., "High-speed high-security signatures," *Journal of Cryptographic Engineering*, 2012.